



湖南大学
HUNAN UNIVERSITY

第七章 分治 Divide and Conquer

—— 湖南大学计算机学院 ——

期中考试



- 一、判断题 (24分)
 - 二、计算题 (20分) 递归式计算, 主要应用主定理、递归树等知识点
 - 三、算法题1 (16分)
 - 四、算法题2 (15分)
 - 五、算法题3 (15分)
 - 六、算法题4 (10分) 教材上内容扩展
- } 教材上的示例

目录

第一节 分治法的设计思想

第二节 最大子段和

第三节 求解递归式

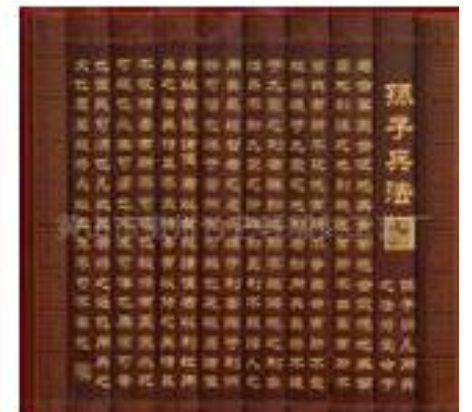
第四节 组合问题中的分治法

第五节 几何问题中的分治法



凡治众如治寡，分数是也；
斗众如斗寡，形名是也。

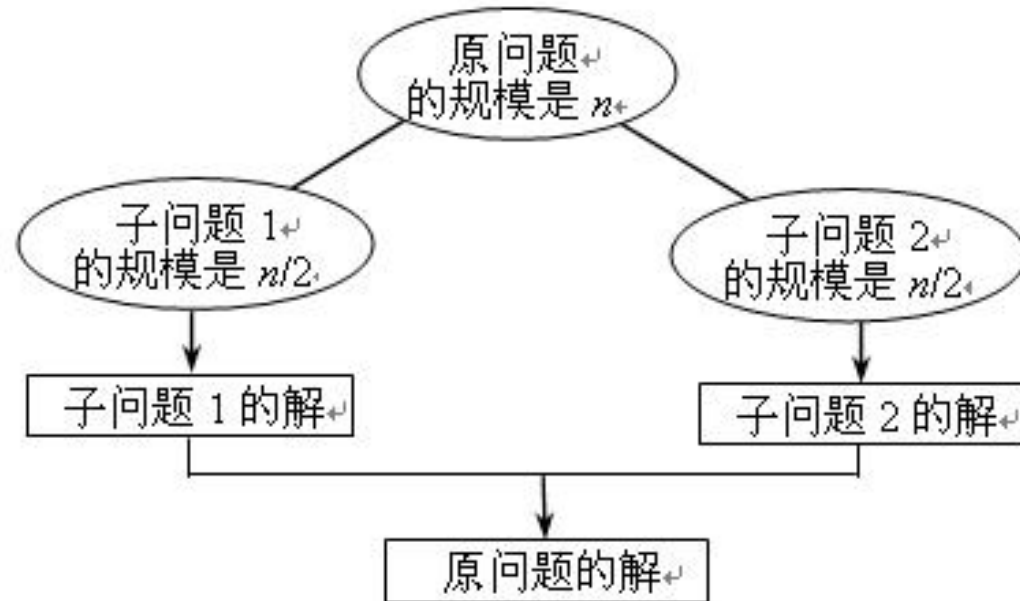
Divide and conquer



分治法的设计思想



将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。



分治法的设计思想



分治法的求解过程是由以下三个阶段组成:

- (1) Divide : 划分
- (2) Conquer : 求解子问题
- (3) Combine : 合并

目录

第一节 分治法的设计思想

第二节 最大子段和

第三节 求解递归式

第四节 组合问题中的分治法

第五节 几何问题中的分治法

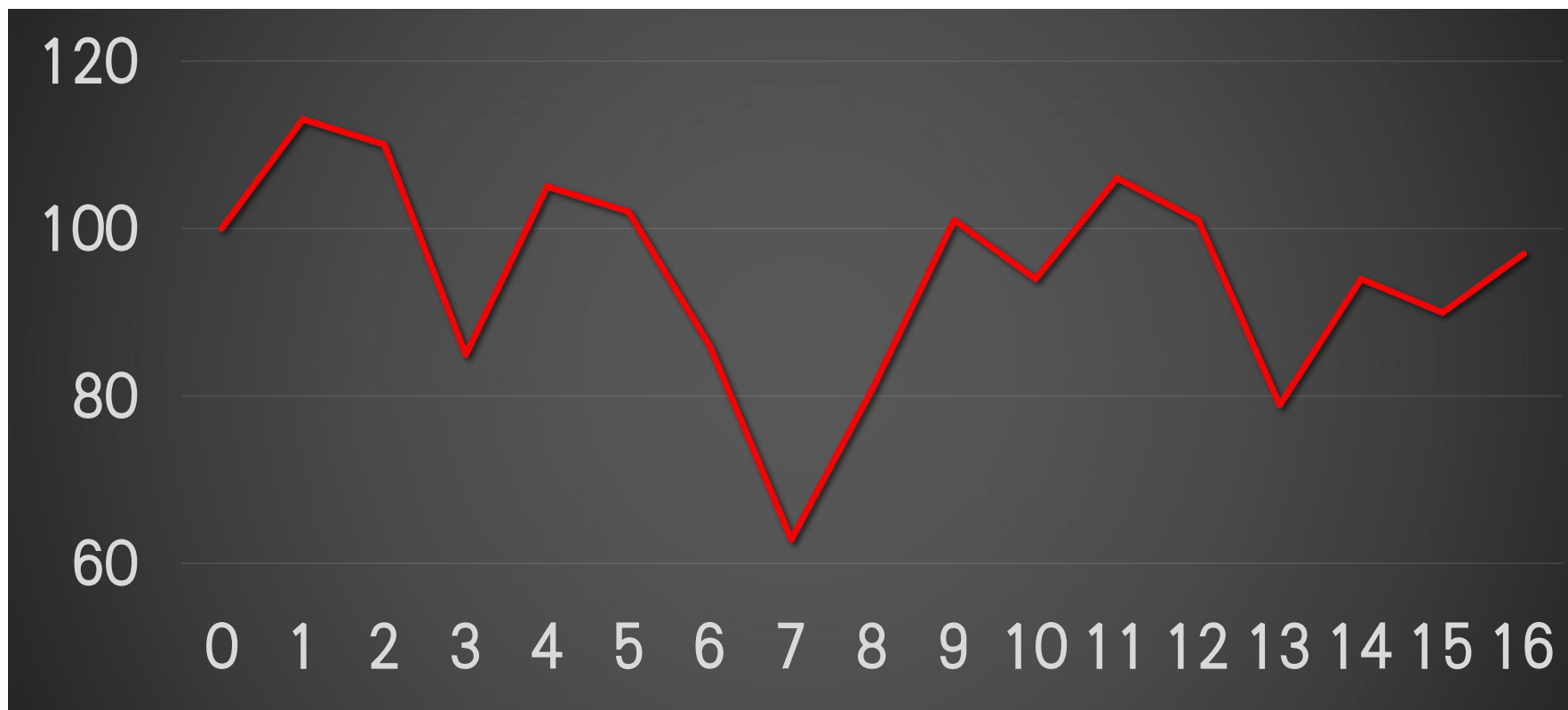
组合问题中的分治法-最大子段和问题



如何购买股票?

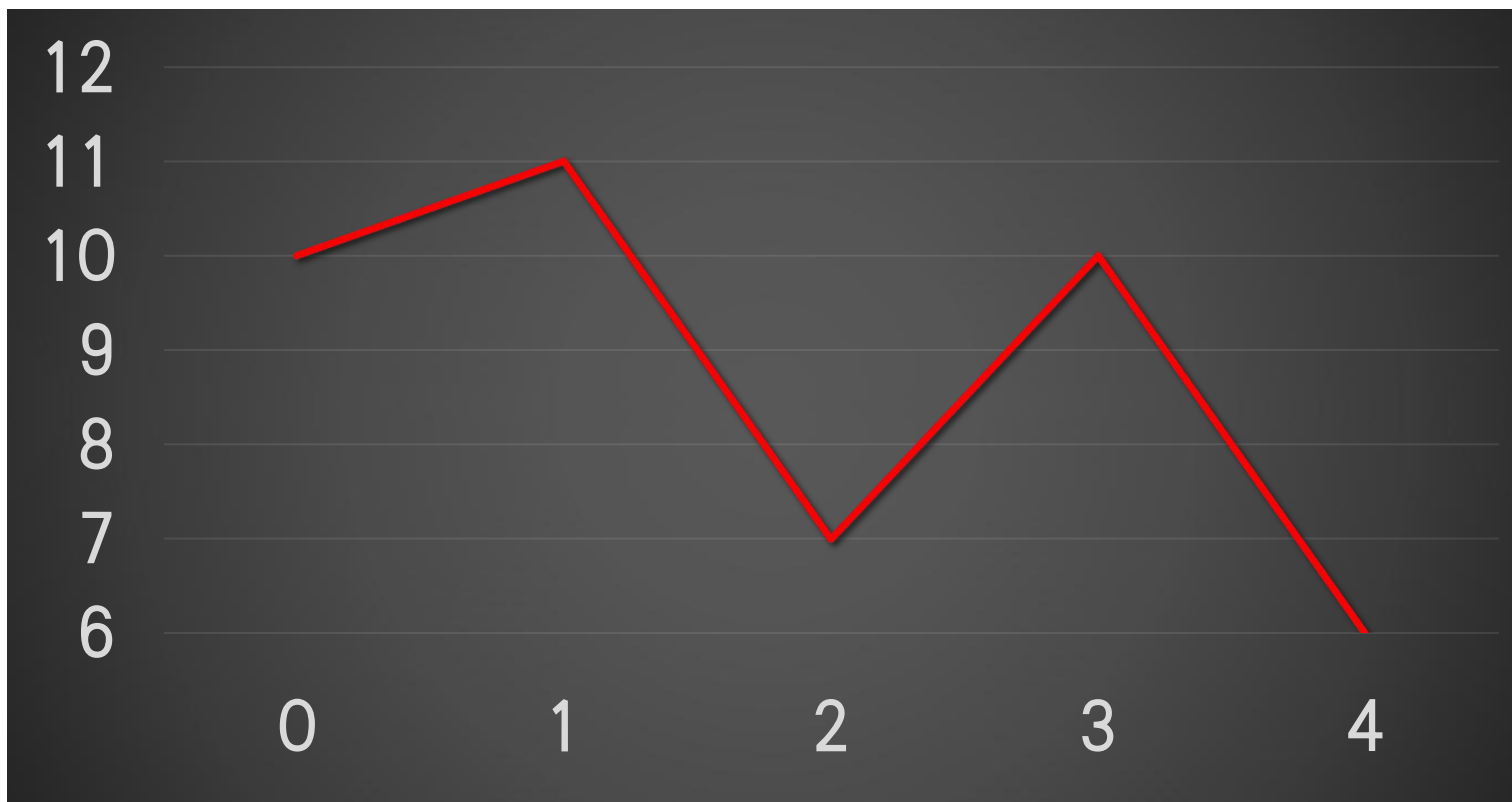


组合问题中的分治法-最大子段和问题



天	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
价格	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
变化		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

组合问题中的分治法-最大子段和问题



天	0	1	2	3	4
价格变化	10	11	7	10	6
		1	-4	3	-4

组合问题中的分治法-最大子段和问题



问题变换(最大子段和问题或最大子数组问题)

给定由 n 个整数（可能有负整数）组成的序列 $(a_1, a_2, \dots, a_n)$ ，最大子段和问题要求找到 $\sum_{k=i}^j a_k$ 的子序列 $(a_i, a_{i+1}, \dots, a_j)$ ($1 \leq i \leq j \leq n$)，当序列中所有整数均为负整数时，其最大子段和为0。例如，序列 $(-20, 11, -4, 13, -5, -2)$ 的最大子段和为 $\sum_{k=2}^4 a_k = 20$ 。

天	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
价格	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
变化		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

组合问题中的分治法-最大子段和问题

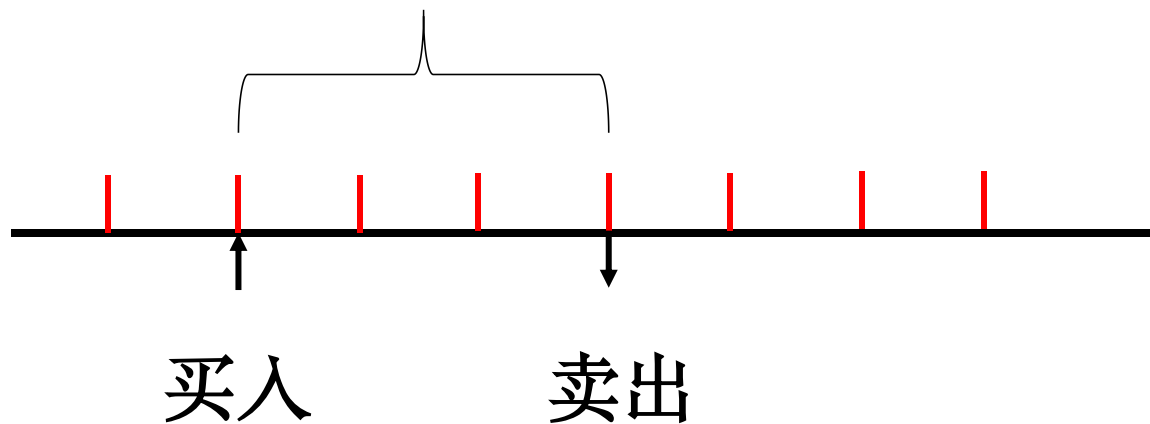


	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

maximum subarray



最简单方法：暴力法解决



时间复杂度分析 $O(n^2)$

暴力法求解的分治优化



- 最优解会是什么样子?

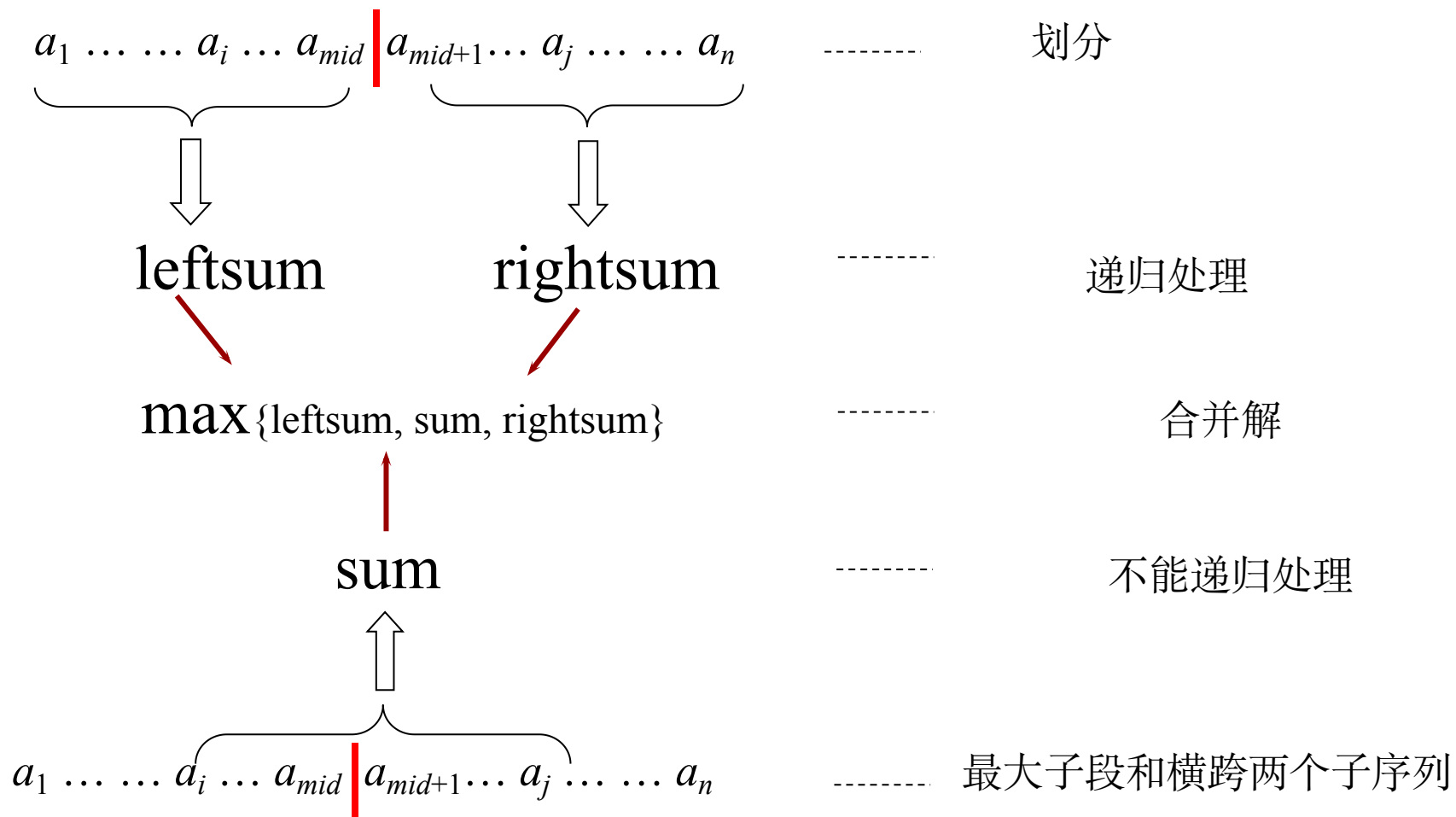
- 不含中间那个元素;
 - 在中间元素左; 在中间元素右
- 包含中间那个元素

$$\underbrace{a_1 \dots \dots a_i \dots a_{mid}} \quad | \quad \underbrace{a_{mid+1} \dots a_j \dots \dots a_n}$$

组合问题中的分治法-最大子段和问题



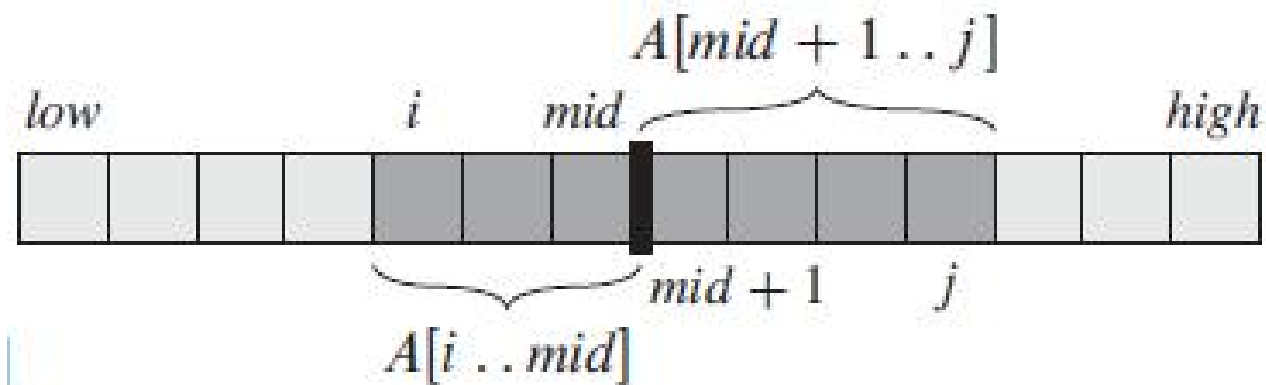
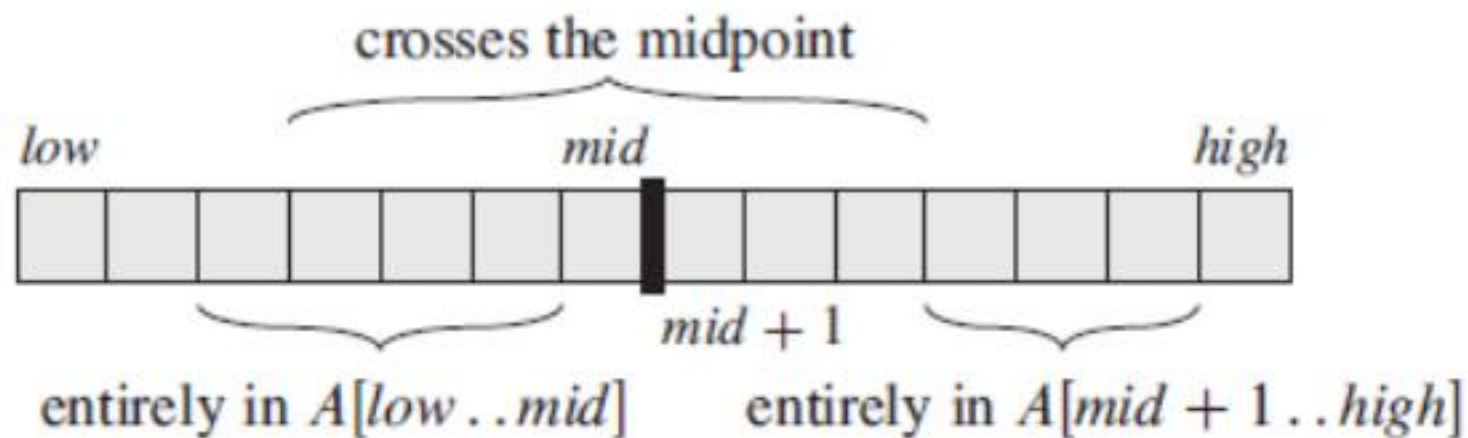
使用分治策略的求解方法



组合问题中的分治法-最大子段和问题



使用分治策略的求解方法



组合问题中的分治法-最大子段和问题



伪代码

FIND-MAX-CROSSING-SUBARRAY ($A, low, mid, high$)

```
1   $left-sum = -\infty$ 
2   $sum = 0$ 
3  for  $i = mid$  downto  $low$ 
4       $sum = sum + A[i]$ 
5      if  $sum > left-sum$ 
6           $left-sum = sum$ 
7           $max-left = i$ 
8   $right-sum = -\infty$ 
9   $sum = 0$ 
10 for  $j = mid + 1$  to  $high$ 
11      $sum = sum + A[j]$ 
12     if  $sum > right-sum$ 
13          $right-sum = sum$ 
14          $max-right = j$ 
15 return ( $max-left, max-right, left-sum + right-sum$ )
```

组合问题中的分治法-最大子段和问题



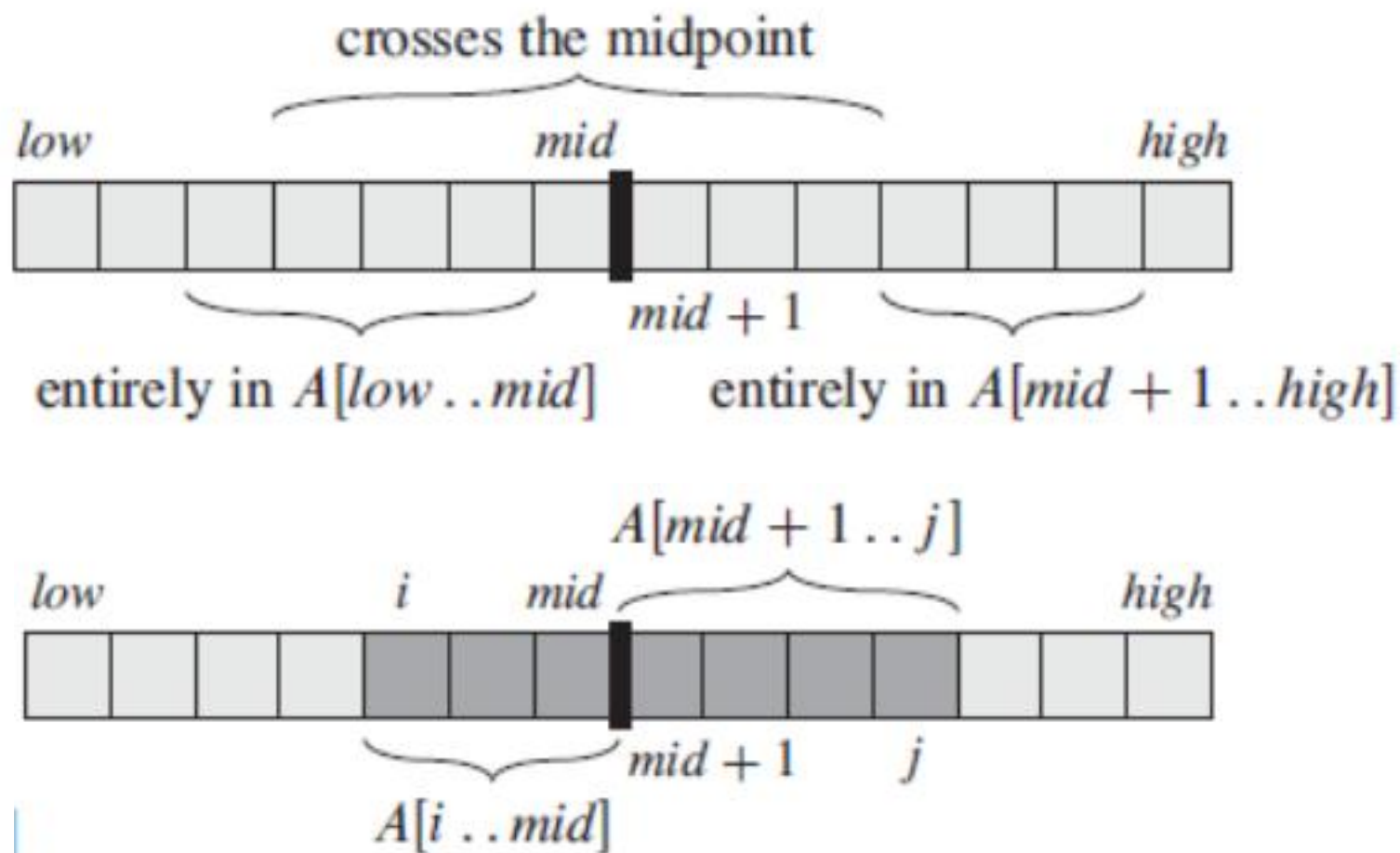
伪代码

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
1  if high == low
2      return (low, high, A[low])           // base case: only one element
3  else mid =  $\lfloor (\textit{low} + \textit{high}) / 2 \rfloor$ 
4      (left-low, left-high, left-sum) =
          FIND-MAXIMUM-SUBARRAY(A, low, mid)
5      (right-low, right-high, right-sum) =
          FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
6      (cross-low, cross-high, cross-sum) =
          FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
7      if left-sum  $\geq$  right-sum and left-sum  $\geq$  cross-sum
8          return (left-low, left-high, left-sum)
9      elseif right-sum  $\geq$  left-sum and right-sum  $\geq$  cross-sum
10         return (right-low, right-high, right-sum)
11     else return (cross-low, cross-high, cross-sum)
```

组合问题中的分治法-最大子段和问题



分治法时间复杂度分析



最大子段和



时间复杂度: $T(n)=2T(n/2)+\Theta(n)=\Theta(n\log n)$

目录

第一节 分治法的设计思想

第二节 最大子段和

第三节 求解递归式

第四节 组合问题中的分治法

第五节 几何问题中的分治法



三种求解递归式的方法:

1. 代入法: 猜测解并用归纳法证明
2. 递归树法: 直观展示代价累加
3. 主方法: 适用于特定形式的“万能公式”

代入法



核心思想:

步骤 1: 猜测: 根据经验或递归树猜测解的形式 (如 $O(n^k)$) 。

步骤 2: 证明: 使用数学归纳法验证猜测是否正确。

- 归纳假设: 假设对于所有 $k < n$, 猜想成立。
- 归纳步骤: 证明对于 n , 猜想也成立。

代入法



求解: $T(n)=4T(n/2)+n$

猜测: $T(n)=O(n^3)$

证明: 数学归纳法

假设对于 $k < n$, $T(k) \leq ck^3$

于是, $T(n)=4T(n/2)+n \leq 4c(n/2)^3+n=cn^3/2+n=cn^3-(cn^3/2-n)$

当 $c > 1$ 且 $n \geq 1$ 的时候, $(cn^3/2-n) \geq 0$, 进而有 $cn^3-(cn^3/2-n) \leq cn^3$

代入法



求解: $T(n)=4T(n/2)+n$

猜测: $T(n)=O(n^2)$.

证明: 数学归纳法

假设对于 $k < n$, $T(k) \leq ck^2$,

那么 $T(n) \leq 4c(n/2)^2 + n = cn^2 + n$

代入法



求解: $T(n)=4T(n/2)+n$

猜测: 假设对于 $k < n$, $T(k) \leq c_1 k^2 - c_2 k$

于是, $T(n)=4(c_1(n/2)^2 - c_2 n/2) + n = c_1 n^2 - 2c_2 n + n = c_1 n^2 - c_2 n - (c_2 - 1)n$

当 $c_2 \geq 1$ 的时候, $(c_2 - 1)n \geq 0$, 进而有 $T(n) \leq c_1 n^2 - c_2 n$

递归树法



核心思想：

可视化展开：将递归调用过程画成一棵树。

- 节点：代表某一层递归调用的非递归代价（即 $f(n)$ 部分）。
- 边：代表递归调用的关系。

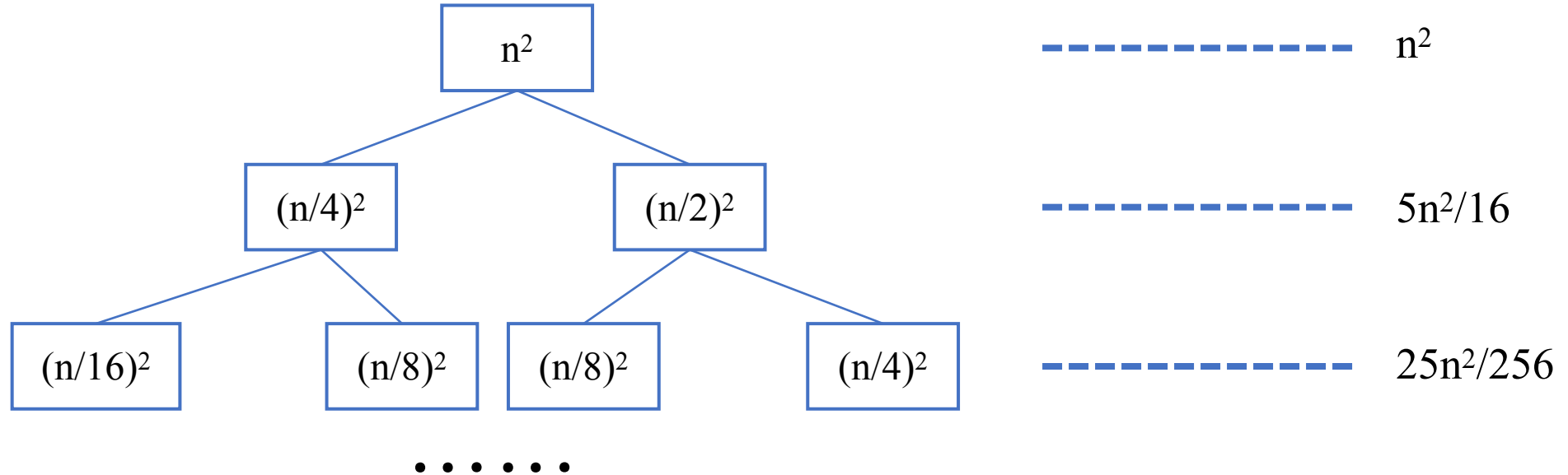
求解步骤：

- 展开：画出树的前几层，找出规律。
- 求和：计算每一层的总代价。
- 累加：将所有层的代价相加（通常形成等比数列或等差数列）。

递归树法



- 例子: $T(n) = T(n/4) + T(n/2) + n^2$



$$T(n) \leq n^2(1 + 5/16 + (5/16)^2 + \dots + (5/16)^k + \dots) < 2n^2 = O(n^2)$$

求解递归式-主方法



主方法适用于下面的递归形式:

$$T(n) = aT(n/b) + f(n)$$

其中, $a \geq 1, b > 1, f$ 为正的渐进函数

求解递归式-主方法



1. 若对某个常数 $\varepsilon > 0$ 有 $f(n) = O(n^{\log_b a - \varepsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$ 。
2. 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \lg n)$ 。
3. 若对某个常数 $\varepsilon > 0$, 有 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 且对某个常数 $c < 1$ 和足够大的 n 有 $af(n/b) \leq cf(n)$, 则 $T(n) = \Theta(f(n))$ 。

求解递归式-主方法



$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

$$T(n) = 3T\left(\frac{n}{4}\right) + n \log n$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

目录

第一节 分治法的设计思想

第二节 最大子段和

第三节 求解递归式

第四节 组合问题中的分治法

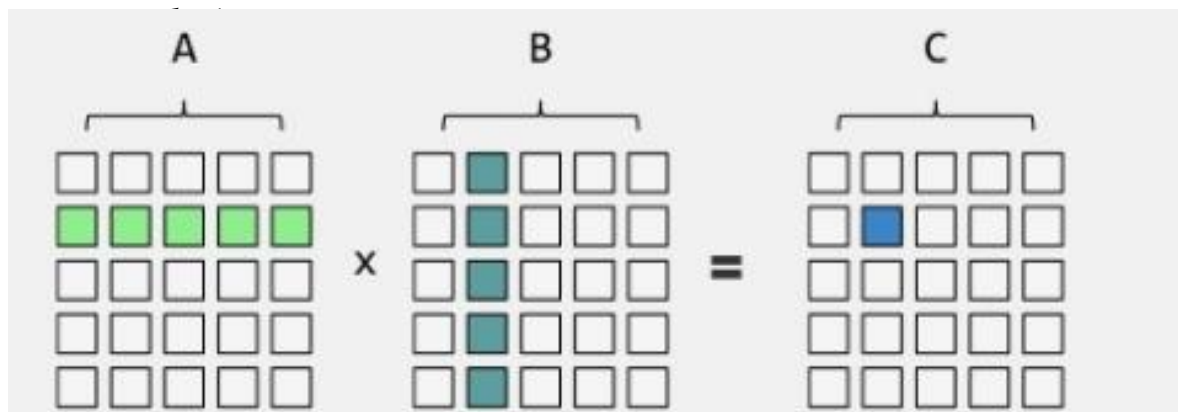
第五节 几何问题中的分治法

组合问题中的分治法-矩阵乘法



$$S(i, j) = A(i, j) + B(i, j) \quad 1 \leq i, j \leq n$$

$$C(i, j) = \sum_{k=1}^n A(i, k) * B(k, j) \quad 1 \leq i, j \leq n$$



矩阵加法运算的时间复杂度是： $\Theta(n^2)$

而矩阵乘法的时间复杂度是： $\Theta(n^3)$

求每个元素 $C(i, j)$ 都需要 n 次乘法和 $n-1$ 次加法运算，且 C 共有 n^2 个元素。

组合问题中的分治法-矩阵乘法



用分治法解决矩阵的乘法问题，先假定 n 是2的 k 次幂，即 $n=2^k$ 。

1) 第一步：递归维度半分。

如果 $n=2$ ，则递归结束。

如果 $n>2$ ，将A 和B 两个 n 维方阵各分解为4 份，每份 $n/2$ 维。当子矩阵的阶还是大于2时，继续将子矩阵分块，直到子矩阵的阶降为2。

第一步将会产出很多个2阶的方阵!

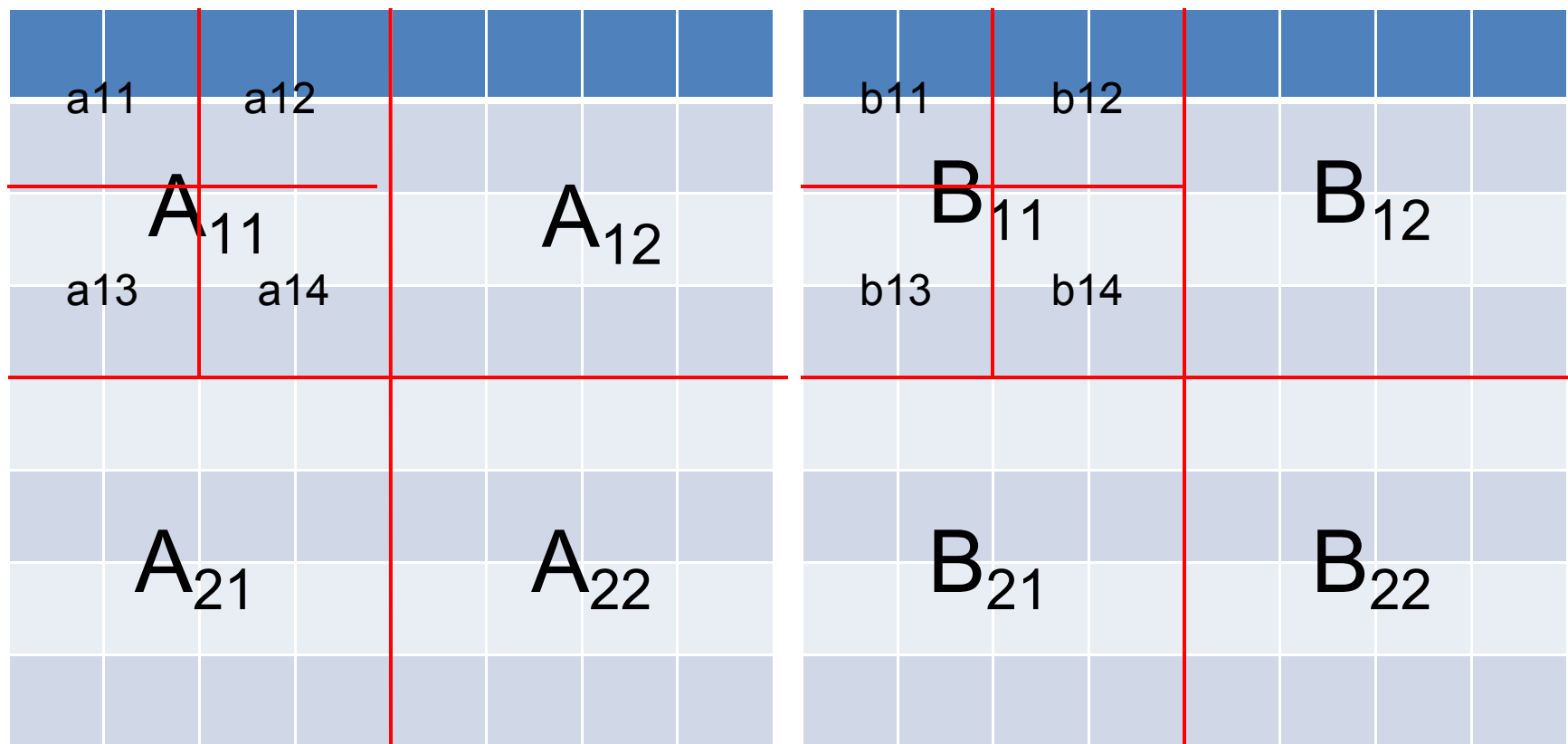
下面用一个动画来展示分矩阵的过程

$$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

A矩阵

B矩阵



组合问题中的分治法-矩阵乘法



2) 第二步：计算两个2阶方阵的乘积之后再合并。

假如A，B两个矩阵是由第一步得出的维数为2的方阵，则：

$$A * B = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} & C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} & C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

通过上面4条式可以将A*B的值存到C矩阵中。



分治法求矩阵乘法的效率分析

如果用 $T(n)$ 记两个 n 阶矩阵相乘所用的时间，则有如下递归关系式：

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$$

$$T(n) = O(n^3)$$

组合问题中的分治法-矩阵乘法



Strassen矩阵乘法

Strassen提出了一种新的算法来计算2个2阶方阵的乘积。他的算法只用了7次乘法运算，但增加了加、减法的运算次数。这7次乘法是：

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

$$M_3 = (A_{21} + A_{22})B_{11}$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

组合问题中的分治法-矩阵乘法



Strassen矩阵乘法

做了7次乘法后，再做若干次加、减法：

$$C_{11}=M_5+M_4-M_2+M_6$$

$$C_{12}=M_1+M_2$$

$$C_{21}=M_3+M_4$$

$$C_{22}=M_5+M_1-M_3-M_7$$

组合问题中的分治法-矩阵乘法



Strassen算法效率分析

Strassen矩阵乘积分治算法中，用了7次对于 $n/2$ 阶矩阵乘积的递归调用和18次 $n/2$ 阶矩阵的加减运算。由此可知，该算法的所需的计算时间 $T(n)$ 满足如下的递归方程：

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + an^2 & n > 2 \end{cases}$$

$$T(n) = O(n^{\log_2 7})$$

目录

第一节 分治法的设计思想

第二节 最大子段和

第三节 求解递归式

第四节 组合问题中的分治法

第五节 几何问题中的分治法

最近点对问题



设 $p_1=(x_1,y_1), p_2=(x_2,y_2), \dots, p_n=(x_n,y_n)$, 是平面上 n 个点构成的集合 S , 最近对问题就是找出集合 S 中距离最近的点对。



你想到些什么？

最近点对问题



最近点对问题的分治策略是:

(1)划分: 将集合 S 分成两个子集 S_1 和 S_2 , 设集合 S 的最近点对是 p_i 和 p_j ($1 \leq i, j \leq n$), 则会出现以下三种情况:

① p_i 和 p_j 在 S_1

② p_i 和 p_j 在 S_2

③ p_i 和 p_j 分别在 S_1 和 S_2

最近点对问题



(2)求解子问题：对于划分阶段的情况①和②可递归求解，如果最近点对分别在集合 S_1 和 S_2 中，问题就比较复杂了。

(3)合并：比较在划分阶段三种情况下最近点对，取三者之中较小者为原问题的解。

最近点对问题



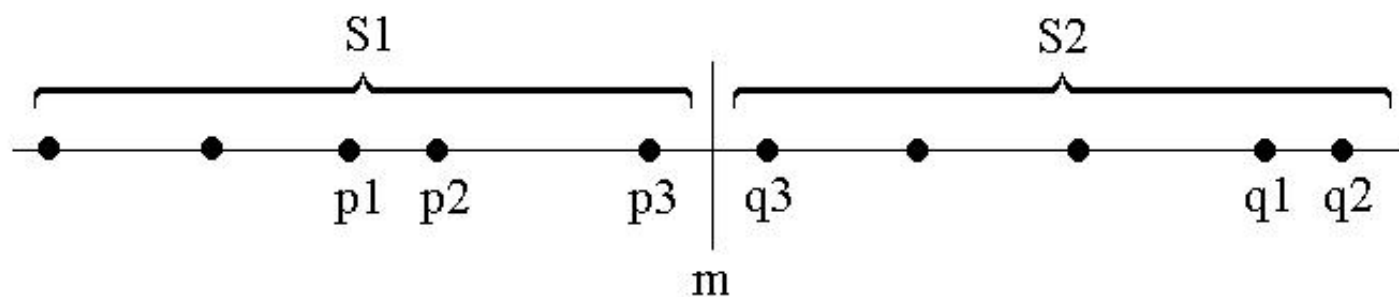
◆ 一维空间

◆ 二维空间 (平面)

最近点对问题



一维空间

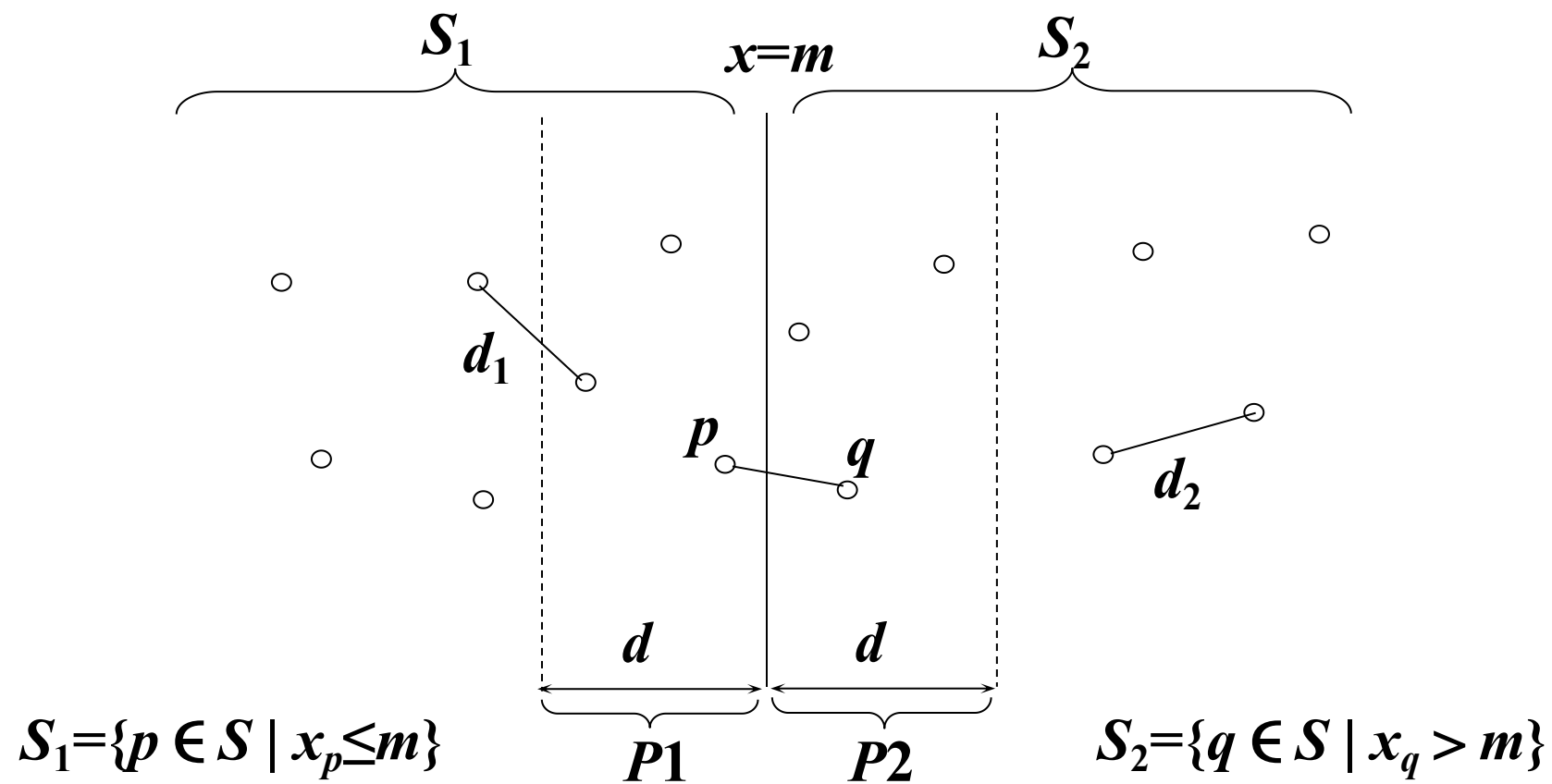


$$T(n)=2T(n/2)+O(1)=O(n)$$

最近点对问题



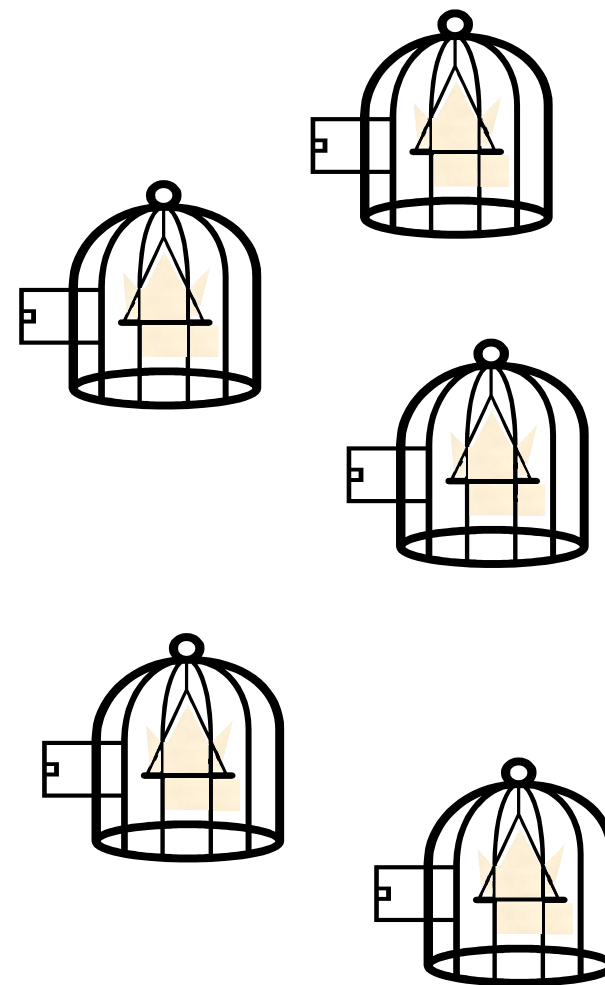
二维空间



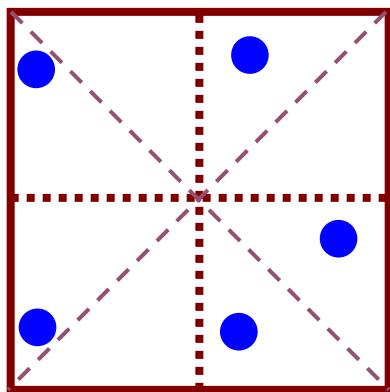
最近点对问题



鸽笼原理



最近点对问题

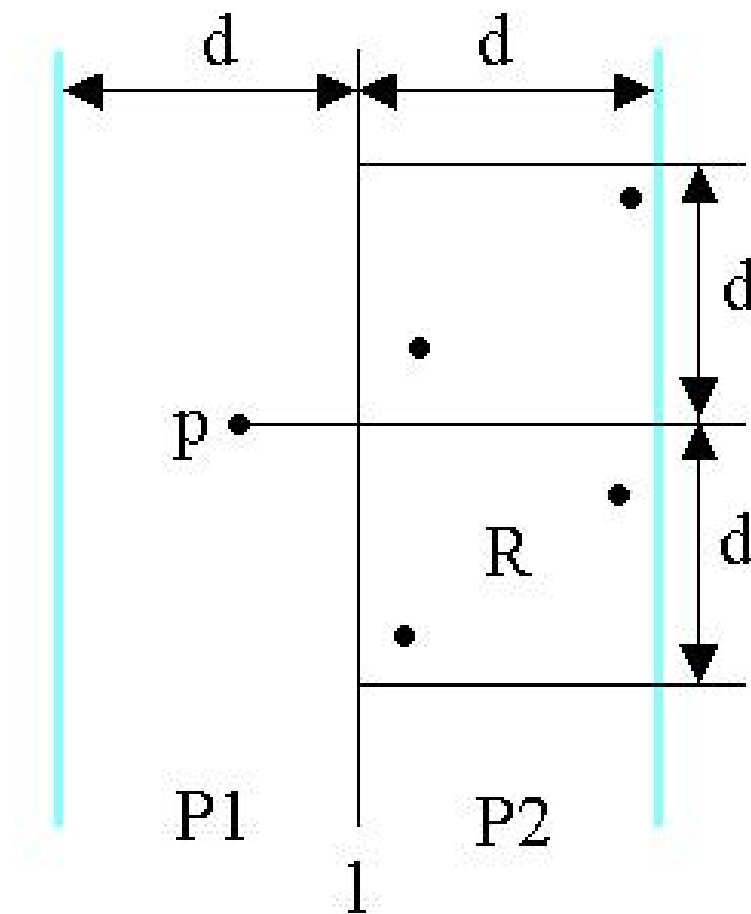


证明：把5个顶点放到边长为2的正方形中，至少存在两个顶点，它们之间的距离小于或等于 $\sqrt{2}$ 。

最近点对问题



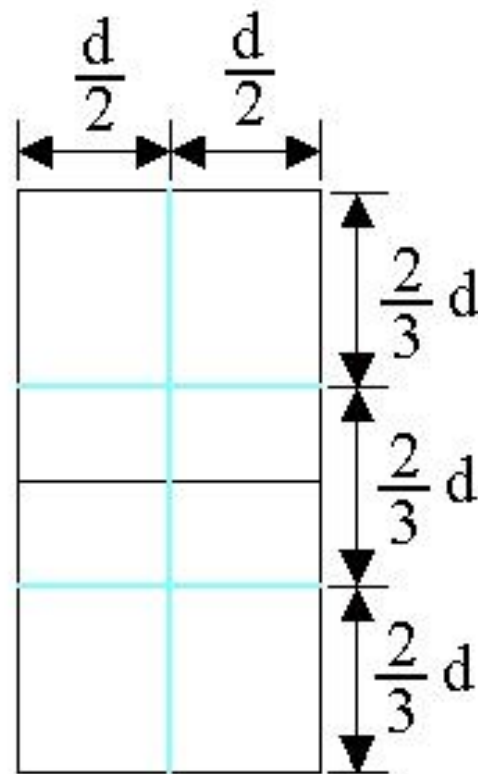
二维空间



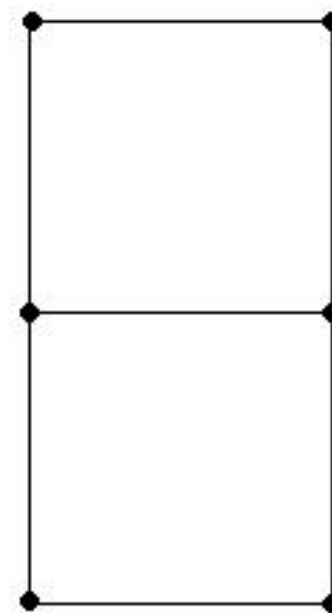
最近点对问题



二维空间



(a)



(b)

最近点对问题



- (1) 代码分析
- (2) 算法时间复杂度分析

$$T(n)=2T(n/2)+O(n)=O(n\log n)$$

峰值查找



- 问题：通过采样在地形中找到一个局部最小值或最大值



一维峰值查找



- 给定一个数组 $A[0..n-1]$:



- $A[i]$ 是一个峰值 如果它不小于它自己邻居:

$$A[i-1] \leq A[i] \geq A[i+1]$$

我们可以想象

$$A[-1] = A[n] = -\infty$$

- 目标: 找到任何峰值

暴力求解峰值查找



- 测试所有元素的峰值

```
for  $i$  in range( $n$ ):  
    if  $A[i - 1] \leq A[i] \geq A[i + 1]$ :  
        return  $i$ 
```

$\left. \begin{array}{l} \text{if } A[i - 1] \leq A[i] \geq A[i + 1]: \\ \text{return } i \end{array} \right\} O(1) \left. \right\} O(n)$

A:

1	2	6	5	3	7	4
---	---	---	---	---	---	---

0 1 2 3 4 5 6

算法 1½



$\text{max}(A)$ 全局最大值是局部最大值

```
 $m = 0$ 
```

```
for  $i$  in range(1,  $n$ ):
```

```
    if  $A[i] > A[m]$ :
```

```
         $m = i$ 
```

```
return  $m$ 
```

$\left. \begin{matrix} \text{if } A[i] > A[m]: \\ \quad m = i \end{matrix} \right\} \Theta(1) \left\} \Theta(n)$

A:

1	2	6	5	3	7	4
---	---	---	---	---	---	---

0 1 2 3 4 5 6

一个思路



查看任何元素 $A[i]$ 及其邻居 $A[i-1]$ 和 $A[i+1]$

如果是峰值：返回 i

否则：在某些侧面上局部上升

- 必须在那个方向上有一个峰值
- 所以可以丢弃数组的其余部分，留下 $A[:i]$ 或 $A[i+1:]$

• Look at any element $A[i]$ and its neighbors $A[i-1]$ & $A[i+1]$

– If peak: return i

– Otherwise: locally rising on some side

- Must be a peak in that direction
- So can throw away rest of array, leaving $A[:i]$ or $A[i+1:]$



一个思路



- 找到一个包含至少一个大于边界的元素的子数组
 - 找到第一个、最后一个和中心元素的最大元素
 - 如果它是峰值，返回它；否则，找到包含其较大邻居的子数组



- 在子数组中递归

一维峰值查找分析



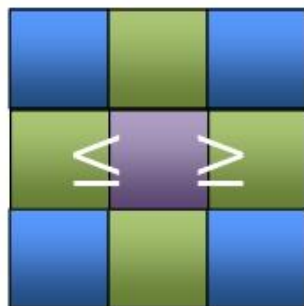
- 将问题划分为大小为 $n/2$ 的 1 个问题
- 划分成本 $O(1)$
- 合并成本 $O(1)$
- 递归关系 $T(n) = T(\frac{n}{2}) + O(1)$

二维峰值查找



$n \times n$

- 给定一个数字矩阵
- 希望一个条目不小于它的 (最多) 4个邻居:



5	4	5	6	4	9	8
3	2	3	1	4	0	3
9	3	9	3	2	4	8
7	6	3	1	3	2	3
9	0	6	0	4	6	4
8	9	8	0	5	3	0
2	1	2	1	1	1	1

二维峰值查找



- 峰值查找: 2D 考虑一个 2D 数组 $A[1\dots n, 1\dots m]$:

10	8	5
3	2	1
7	13	4
6	8	3

- 如果一个元素 $A[i]$ 不小于它的 (最多 4 个) 邻居, 那么它就是 2D 峰值。
- 问题: 找到一个 2D 峰值。

算法 I: 使用 1D 算法



算法 I:

- 对于每一列 j , 找到它的全局最大值 $B[j]$
- 对 $B[1...m]$ 应用 1D 峰值查找器以找到一个峰值 (例如 $B[j]$)
- 运行时间?
 - ..是 $O(n \cdot m)$
- 正确性:
 - $B[j]$ 不小于 $B[j-1]$ 、 $B[j+1]$
 - 对于任何 k , $B[k]$ 不小于来自 A 的第 k 列的任何元素
 - 因此, $B[i]$ 不小于来自 A 的列 $j-1$ 、 j 和 $j+1$ 的任何元素
 - 但这包括 A 中 $B[j]$ 的所有邻居, 所以 $B[j]$ 是 A 中的一个峰值

12	8	5
11	3	6
10	9	2
8	4	1
12	9	6

算法 I: 使用 1D 算法



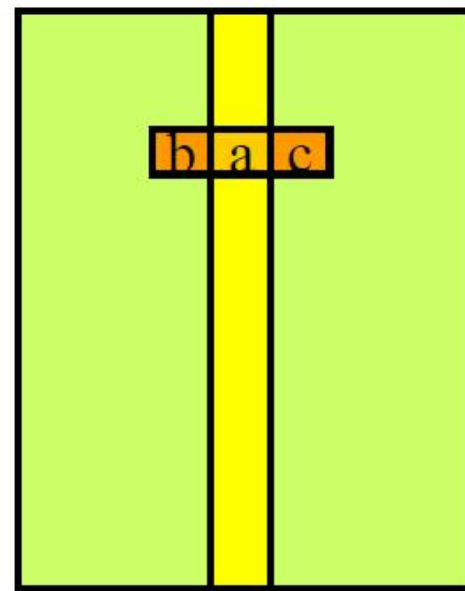
- 观察: 1D 峰值查找器仅使用 $O(\log m)$ 个 B 条目
- 我们可以修改算法 I, 使其仅在需要时计算 $B[i]$!
- 总时间?
 - 只有 $O(n \log m)$!
 - 需要 $O(\log m)$ 个条目 $B[j]$ -
 - 每个在 $O(n)$ 时间内计算

12	8	5
11	3	6
10	9	2
8	4	1
12	9	6

算法 II



- 选择中间列 ($j=m/2$)
- 在该列中找到全局最大值 $a=A[i,m/2]$ (如果 $m=1$, 则退出)
- 将 a 与 $b=A[i,m/2-1]$ 和 $c=A[i,m/2+1]$ 进行比较
- 如果 $b>a$, 则在左侧列上递归
- 否则, 如果 $c>a$, 则在右侧列上递归
- 否则 a 是 2D 峰值!



算法 II 例子



- 选择中间列 ($j=m/2$)
- 在该列中找到全局最大值 $a=A[i,m/2]$ (如果 $m=1$, 则退出)
- 将 a 与 $b=A[i,m/2-1]$ 和 $c=A[i,m/2+1]$ 进行比较
- 如果 $b>a$, 则在左侧列上递归
- 否则, 如果 $c>a$, 则在右侧列上递归
- 否则 a 是 2D 峰值!

12	8	5
11	3	6
10 _b	9 _a	2 _c
8	4	1

算法 II 正确性



- 声明：如果 $b > a$ ，那么在左侧列中存在一个峰值
- 证明（通过矛盾）：
 - 假设左侧没有峰值
 - 那么 b 必须有一个具有更高值的邻居 b_1
 - 并且 b_1 必须有一个具有更高值的邻居 b_2
 - 一 ...
 - 我们必须留在左侧 - 为什么？
 - （因为我们不能进入中间列）
 - 但有时，我们会用尽左侧列的元素
 - 因此，我们必须在某处找到一个峰值

12 _{b2}	8	5
11 _{b1}	3	6
10 _b	9 _a	2
8	4	1

算法II复杂度



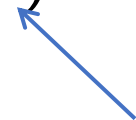
我们有

递归



$$T(n, m) = T(n, m/2) + O(n)$$

扫描中间列



因此

$$T(n, n) = O(n) + O(n) + \cdots O(n)$$



$\log_2 m$

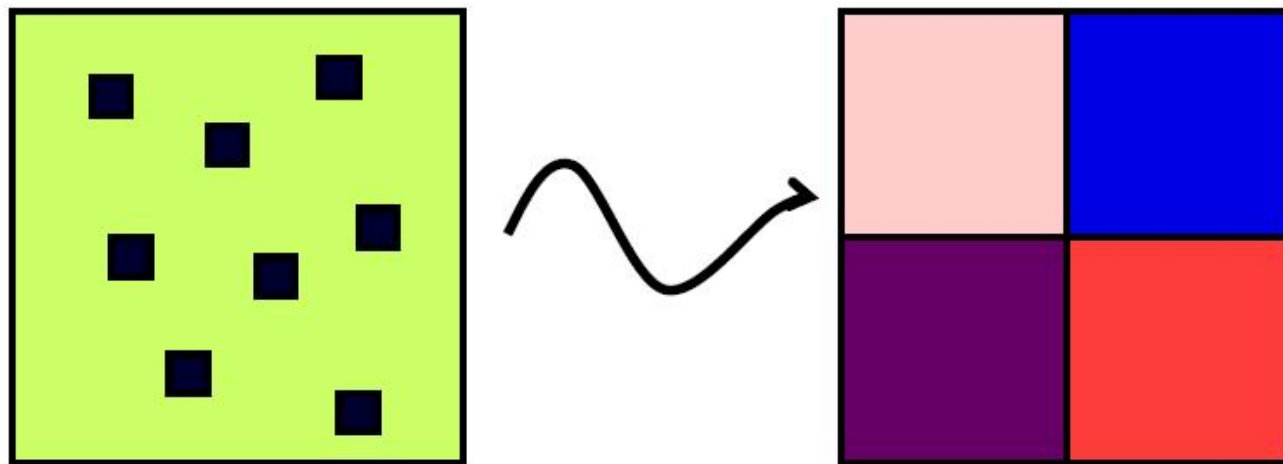
比 $O(n \log m)$ 快?



- 想法:

只读取 $O(n+m)$ 个元素, 将 $n \times m$ 个候选者的数组减少到 $n/2 \times m/2$ 个候选者的数组

- 图形化:

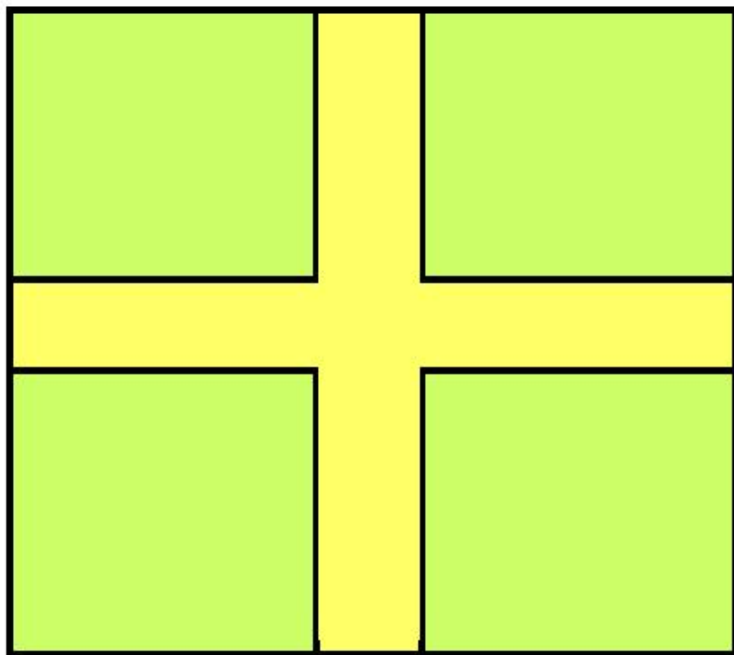


只读取 $O(n+m)$ 个元素

朝着线性算法前进



哪些元素值得检查？

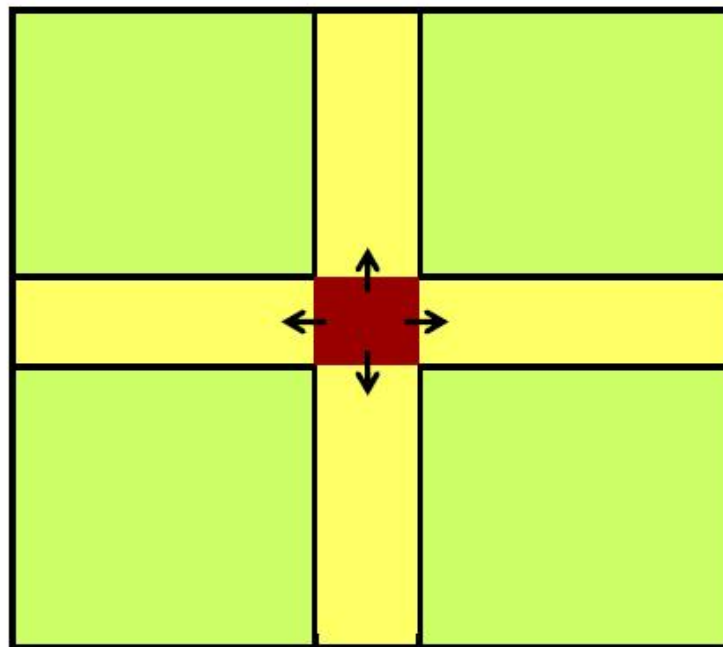


假设我们在十字架上找到全局最大值

朝着线性算法前进



哪些元素值得检查?



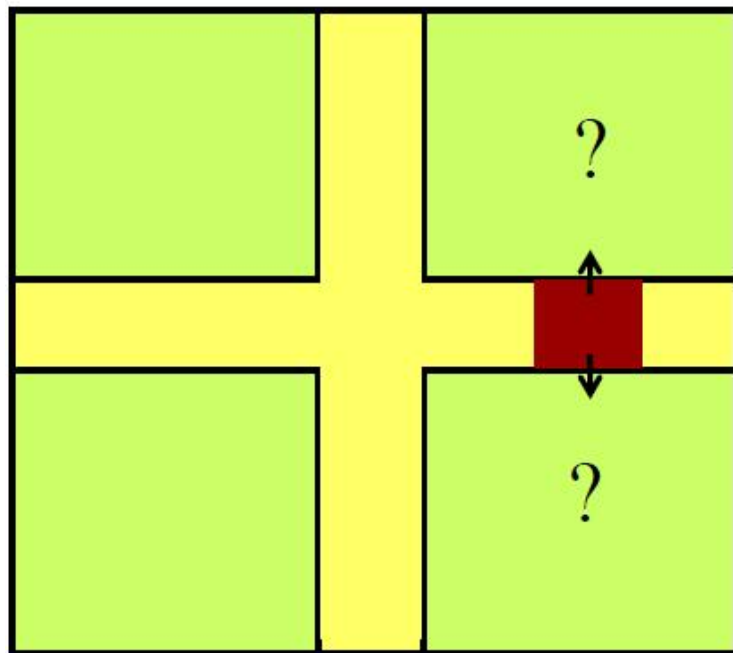
假设我们在十字架上找到全局最大值

如果中间元素完成!

朝着线性算法前进



哪些元素值得检查？



- 假设我们在十字架上找到全局最大值
- 如果中间元素完成!
- 否则两个候选子正方形
- 通过查看不在十字架上的邻居来确定要选择的子正方形 (如算法 II)

声明: 上述过程选择的子正方形 (如果有的话) 总是包含大正方形的峰值。
但是: 声明 2: 所选子正方形的并非每一个峰值都一定是大正方形的峰值。
因此, 很难递归。

分治法#0



看中心元素并没有把问
题分成几块

5	4	5	6	4	9	8
3	2	3	1	4	0	3
9	3	9	3	2	4	8
7	6	3	1	3	2	3
9	0	6	0	4	6	4
8	9	8	0	5	3	0
2	1	2	1	1	1	1

分治法#0



看中心元素并没有把问
题分成几块

5	4	5	6	4	9	8
3	2	3	1	4	0	3
9	3	9	3	2	4	8
7	6	3	1	3	2	3
9	0	6	0	4	6	4
8	9	8	0	5	3	0
2	1	2	1	1	1	1

- [illegible]

正确性



- **定理：** 如果你进入一个象限，它包含整个数组的峰值 [爬上去]
- **不变量：** 当我们递归下降时，窗口的最大元素永远不会减少
- **定理：** 已访问的象限中的峰值也是整体数组中的峰值

0	0	0	0	0	0	0	0	0
0	5	4	5	6	4	4	8	0
0	3	2	3	1	4	4	3	0
0	9	3	9	3	2	2	8	0
0	7	6	3	1	3	3	3	0
0	9	0	6	0	4	4	4	0
0	8	9	8	0	5	5	0	0
0	2	1	2	1	1	1	1	0
0	0	0	0	0	0	0	0	0

→ proofs in recitation

复杂度分析



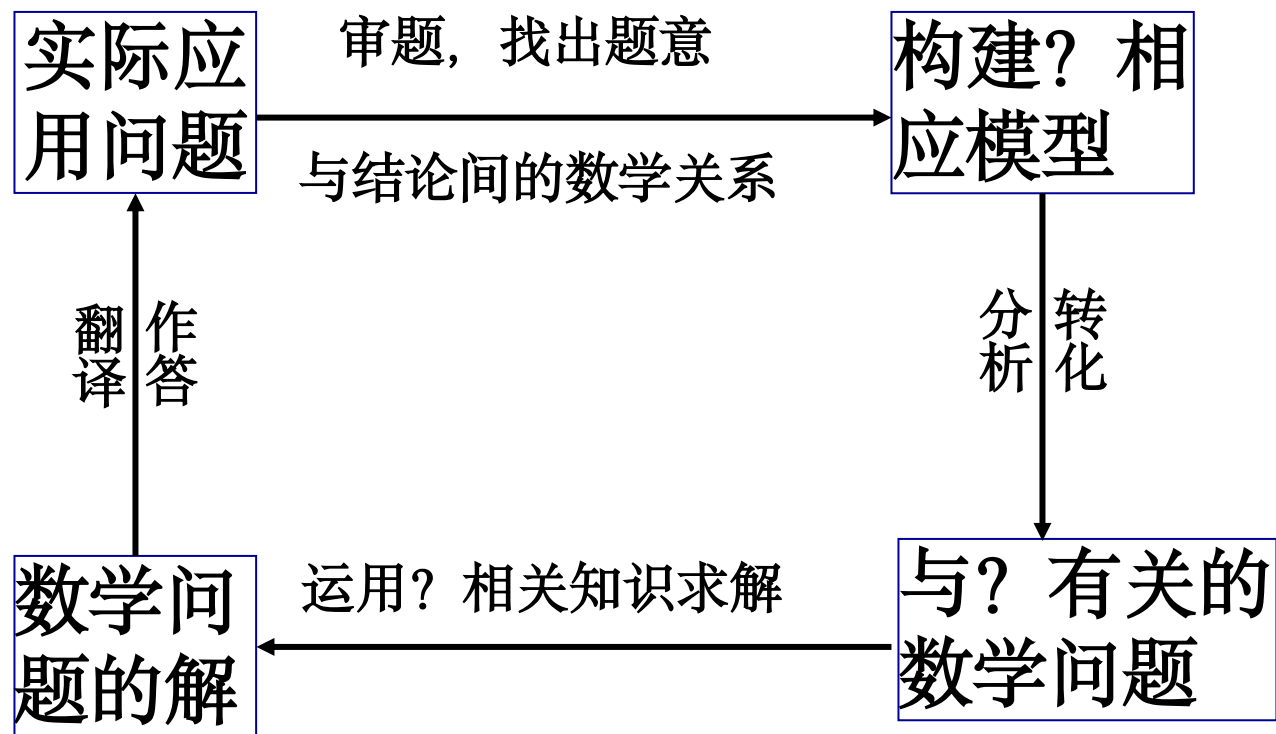
假设算法具有以下递归关系:

$$T(n, m) = T\left(\frac{n}{2}, \frac{m}{2}\right) + \Theta(n + m)$$

因此

$$\begin{aligned} T(n, m) &= \Theta(n + m) + \Theta\left(\frac{n + m}{2}\right) \\ &\quad + \Theta\left(\frac{n + m}{4}\right) \\ &\quad + \dots + \Theta(1) \\ &= \Theta(n + m) \quad ! \end{aligned}$$

小结





湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——